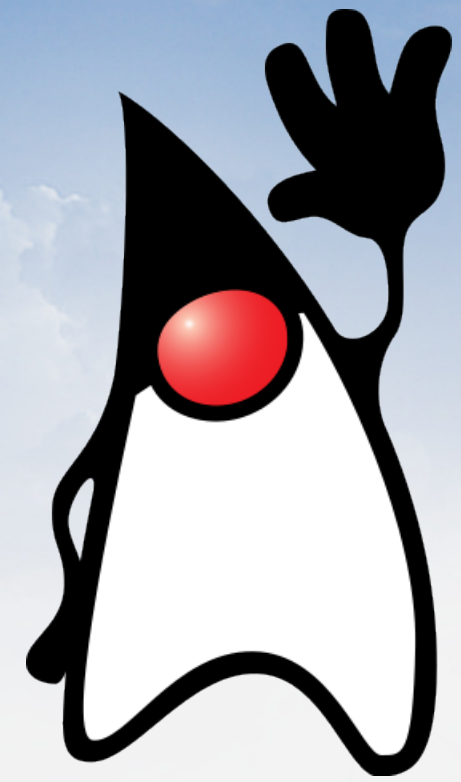




Java Iterator and Enum

COP 3330
Department of Computer Science
University of Central Florida
Dr. Bacanli

One confession

Every class that implements **Iterable** interface appropriately, can be traversed with short hand syntax.

We were kind of using iterator before

```
for(Item item : itemArrayList) {  
    // do stuff  
    //itemArrayList.remove(item);  
}
```

We canNOT remove or add with this syntax.

Is there a way to remove while traversing?

Search and remove elements

```
package arrTest;
import java.util.ArrayList;
class arrTest {
    public static void removeEls(ArrayList<Integer> arr, int num)
    {
        for(int i=0; i<arr.size(); i++) {
            if(arr.get(i) == num) {
                arr.remove(i);
            }
        }
    }

    public static void main(String[] args) {
        ArrayList<Integer> arrlist = new ArrayList<>();
        arrlist.add(1);arrlist.add(2);arrlist.add(3);
        arrlist.add(1);arrlist.add(1);arrlist.add(3);
        removeEls(arrlist,1); // remove all 1s
        System.out.println(arrlist);
    }
}
```

Remove all occurrence
of an element in the
ArrayList

[2, 3, 1, 3]

Size of the ArrayList
changes when we
remove an element
from the ArrayList

Search and remove elements

```
package arrTest;
import java.util.ArrayList;
class arrTest {
    public static void removeEls(ArrayList<Integer> arr, int num)
    {
        for(int i = arr.size() - 1; i >= 0; i--) {
            if(arr.get(i) == num) {
                arr.remove(i);
            }
        }
    }

    public static void main(String[] args) {
        ArrayList<Integer> arrlist = new ArrayList<>();
        arrlist.add(1);arrlist.add(2);arrlist.add(3);
        arrlist.add(1);arrlist.add(1);arrlist.add(3);
        removeEls(arrlist,1);// remove all 1s
        System.out.println(arrlist);
    }
}
```

Remove all occurrence of an element in the ArrayList

[2, 3, 3]

If you can traverse the list in reverse manner, you won't get effected by size reduction.
Can you implement the same solution but forward traverse?

compact1, compact2, compact3

java.util

Class ArrayList<E>

java.lang.Object

java.util.AbstractCollection<E>

java.util.AbstractList<E>

java.util.ArrayList<E>

All Implemented Interfaces:

Serializable, Cloneable, Iterable<E>, Collection<E>, List<E>,

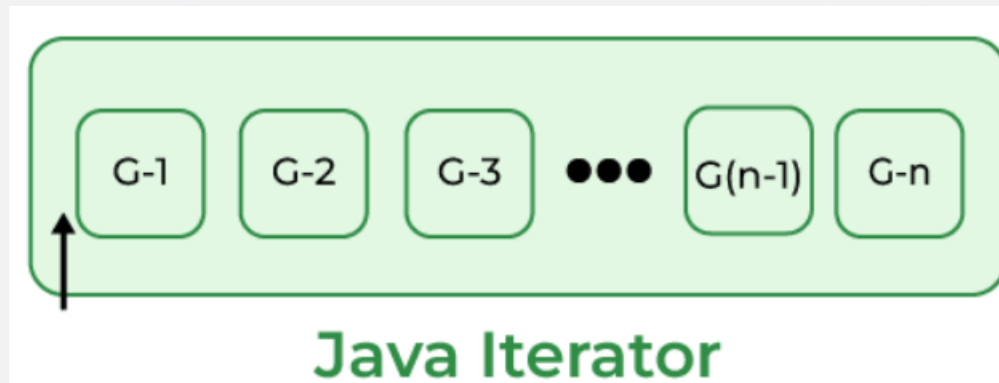
Direct Known Subclasses:

AttributeList, RoleList, RoleUnresolvedList

If an Object implements Iterable<E>, we can traverse it shorthand

Iterator

- It encapsulates the collection object so that we can traverse it in a way the Iterator class has implemented
- Standard Java collections generally implements iterator.
- It works with a cursor traversing through the list
- We were using a similar thing with StringTokenizer
- The cursor only moves forward!



```
package itTest;
import java.util.ArrayList;
import java.util.Iterator;
public class Test {
    public static void main(String[] args)
    {
        ArrayList<Integer> al = new ArrayList<>(10);
        // Iterating over the List
        for (int i = 0; i < 10; i++){
            al.add(i);
        }
        System.out.println(al);
        Iterator<Integer> itr = al.iterator();
        while (itr.hasNext()) {
            int i = itr.next(); //Get the current item
            System.out.print(i + " ");
            if (i % 2 != 0){
                itr.remove();
            }// end of if

        }//end of while
    }//end of main
} //end of class
```

Using Iterator

Iterator methods

- next()
- hasNext()
- remove()

What is the advantage?
for(Integer el:al) ... is also possible?

```
package itTest;
import java.util.*;

public class Test {
    public static void main(String[] args)
    {
        HashMap<String, Integer> hm=new HashMap<>();
        hm.put("Java",34);
        hm.put("Python",99);
        Iterator<Map.Entry<String, Integer>> iterator
            = hm.entrySet().iterator();

        while (iterator.hasNext()) {
            Map.Entry<String, Integer> entry = iterator.next();
            String key = entry.getKey();
            Integer value = entry.getValue();
            System.out.println("Key=" + key + ", Value=" + value);
            //for removal of current element -> iterator.remove();
            //hm.put(key,1); is possible
        }//end of while

    }//end of main
} //end of class
```

Using Iterator

With HashMap

Key=Java, Value=34
Key=Python, Value=99

Do NOT call next()
more than once.
Each next() call will
move the pointer to
the next one.

Iterator

- It gives us opportunity to remove element without reverse traversal
- We can add/modify an element while traversing.
- More clean way to traverse collections
- Uni-directional traversal is possible only.
- Custom iterators can be implemented for complex data structures
 - Using iterator to traverse a binary/non binary tree
 - Tree which has stacks in each node

ListIterator

- It is only applicable for List collection implemented classes like ArrayList, LinkedList, etc.
- It provides **bi**-directional iteration.
- We can move back and forward, remove and modify the element.
- ListIterator must be used when we want to enumerate elements of List type. Something which implements *List*.
- add(E e) : adding an element to that cursor position
- set(E e) : setting an element at that cursor
- previous() and hasPrevious() are for traversing backwards

```
package itTest;
import java.util.ArrayList;
import java.util.ListIterator;
```

```
public class TestArrayList {
```

```
    public static void itEx(ArrayList<Integer> al){
    {
        ListIterator<Integer> ltr = al.listIterator();
        int i=0;
        while (ltr.hasNext()) {
            i = ltr.next();
            System.out.print(i + " ");
            if (i % 2 == 0) {
                ltr.set(i+1); // Change to odd
                ltr.add(-1); // Add -1 after that
            } //end of if

        } //end of while
    } //end of main
}
```

```
} //end of class
```

ListIterator

```
package itTest;
```

```
public class Test {
    public static void main(String[] args)
    {
        ArrayList<Integer> arr;
        arr = new ArrayList<>();
        arr.add(1);
        arr.add(2);
        arr.add(3);
        TestArrayList.itEx(arr);
        System.out.println("\n"+arr);
    } //end of main
} //end of class
```

```
1 2 3
```

```
[1, 3, -1, 3]
```

Java forEach (version 8)

```
public static void main(String[] args) {  
    ArrayList<Integer> al = new ArrayList<Integer>();  
    al.add(1); al.add(2); al.add(3);  
    al.forEach((elem) -> {  
        System.out.println(elem);  
    });  
  
    HashMap<String,Integer> hm = new HashMap<>();  
    hm.put("a", 1); hm.put("b", 2); hm.put("c", 3);  
    hm.forEach( (name,id) -> {  
        System.out.println(name+" "+id);  
        hm.put("a",4);  
    });  
}
```

be aware of this
syntax
forEach and
stream
is also used with
Java>8

Java Enum

```
package oneTester;

public enum Day {
    SUNDAY, MONDAY, TUESDAY,
    WEDNESDAY,
    THURSDAY, FRIDAY, SATURDAY
}
```

Today is MONDAY

SUNDAY MONDAY TUESDAY
WEDNESDAY THURSDAY FRIDAY
SATURDAY

Enums can not have subClasses

```
package oneTester;

public class OneTester {
    public static void main(String[] args) {
        Day d1=Day.MONDAY;
        if(d1==Day.FRIDAY) {
            System.out.println("Day is Friday");
        }//end of if

        System.out.println(d1+"\n");
        Day[] arr=Day.values();
        for(Day d:arr) {
            System.out.print(d+" ");
        }

    }// end of main method
}
```

Static import

```
package OneTester;
import static java.lang.Math.*;
import static java.lang.System.out;

public class OneTester {
    public static void main(String[] args) {
        double sq4 = sqrt(4);
        out.println("Square root of 4 is " +sq4);
    }
}
```

We don't need `Math.sqrt()` anymore. We can use `sqrt` as if it is implemented in our code.

Static import examples can be seen on internet code examples. It is a bad practice. If possible, do not use it!

Only one package

```
package personPackage;

public class PersonTester {

    public static void main(String[] args)
    {
        Person p=new Person();
        System.out.println( p.getAge() );
        // the below code doesn't work
        System.out.println( p.age );
        // age variable is only for Person class
    }
}
```

```
package personPackage;

public class Person {
    private int age;

    public Person(){
        age = 0;
    }

    public int getAge() {
        return age;
    }

    public void setAge(int ageg) {
        age = ageg;
    }
}
```

More than one package

```
package vipPackage;

import personPackage.Person;
// Importing is needed
public class VIPTester {

    public static void main(String[] args)
    {
        Person p=new Person();
        System.out.println( p.getAge() );
        // the above code will work
        // since getAge() is public
        //protected is only for the classes
        // inside the same package
        // System.out.println( p.setAge(99) );
        //The above code won't work
    }
}
```

```
package personPackage;

public class Person {
    private int age;

    public Person(){
        age = 0;
    }

    public int getAge() {
        return age;
    }

    protected void setAge(int ageg) {
        age = ageg;
    }
}
```

More than one package, static call

```
package vipPackage;
```

```
import personPackage.Person;
```

```
// Importing is needed
```

```
public class PersonUse{
```

```
    public static void main(String[] args)
```

```
{
```

```
        Person p=new Person();
```

```
        System.out.println( Person.hi5());
```

```
}
```

```
}
```

```
package personPackage;
```

```
public class Person {
```

```
    private int age;
```

```
    public Person(){
```

```
        age = 0;
```

```
}
```

```
    public static int hi5() {
```

```
        return 5;
```

```
}
```

```
}
```

Here, we are calling a static method inside the Person class.

Since we imported the Person class, we can call that method as long as it is public.

Access Modifiers

- public
- protected
- private
- no modifier
(package-private)

Access Levels				
Modifier	Class	Package	Subclass	World
public	Y	Y	Y	Y
protected	Y	Y	Y	N
<i>no modifier</i>	Y	Y	N	N
private	Y	N	N	N

Source: <https://docs.oracle.com/javase/tutorial/java/javaOO/accesscontrol.html>

only one package

```
package personPackage;
//Same package

public class VIP extends Person {

    public VIP() {
        super();
    }

    public static void main() {
        VIP v = new VIP();
        System.out.println(v.age);
        //this will work
    }
}
```

Child class is inside the same package.

```
package personPackage;

public class Person {
    int age; // default restrictor

    public Person() {
        age = 0;
    }

    public int getAge() {
        return age;
    }

    public void setAge(int ageGiven) {
        age = ageGiven;
    }
}
```

More than one package

```
package vipPackage;
//Different Packages

public class VIP extends Person {

    public VIP() {
        super();
    }

    public static void main() {
        VIP v = new VIP();
        System.out.println(v.age);
        //this will not work
        //age has default restrictor
    }
}
```

```
package personPackage;

public class Person {
    int age; // default restrictor

    public Person() {
        age=0;
    }

    public int getAge() {
        return age;
    }

    public void setAge(int ageGiven) {
        age = ageGiven;
    }
}
```

Import statement is needed to access constructor of the Person class
We are calling the constructor using super.

More than one package

```
package vipPackage;

import personPackage.Person;
//Different Packages

public class VIP extends Person {

    public VIP() {
        super();
    }

    public static void main() {
        VIP v = new VIP();
        System.out.println(v.age);
        //this will not work
    }
}
```

Child class is inside another package.

```
package personPackage;

public class Person {
    int age; // default restrictor

    public Person() {
        age=0;
    }

    public int getAge() {
        return age;
    }

    public void setAge(int ageGiven) {
        age = ageGiven;
    }
}
```

default restrictor is very strict. Even the child class cannot access if it is not inside the same package.

More than one package

```
package vipPackage;
import java.util.*; //useless, OK!
import personPackage.Person;
//Different Packages

public class VIP extends Person {

    public static void main(){
        VIP v = new VIP();
        System.out.println(v.age);
        //this will work
    }
}
```

Child class is inside another package.

protected is more merciful. Even the child class is inside another package, it can access the parent class feature.

```
package personPackage;

public class Person {
    protected int age;

    public Person(){
        age=0;
    }

    public int getAge() {
        return age;
    }

    public void setAge(int ageGiven) {
        age = ageGiven;
    }
}
```